2 Integration Testing (40 marks)

The StreetRace Manager is a command-line system designed to manage underground street races, crew members, vehicles, and missions. The system was built and tested module-by-module to ensure both individual functionality and seamless cross-module data flow.

Core Modules

- **Registration:** Registers new crew members into the system. Each member is given a unique ID upon registration. The member's name is stored at this stage and their role defaults to `unassigned`.
- **Crew Management:** Manages roles and skill levels. A member must be registered before a role can be assigned. Valid roles are: `driver`, `mechanic`, `strategist`, `scout`, and `medic`.
- **Inventory:** Tracks all assets owned by the crew, including cars (name, speed, condition), spare parts, tools, and a cash balance (starting default of $10,000). Car conditions can be `excellent`, `good`, `damaged`, or `wrecked`.
- **Race Management:** Creates race events and manages entries. Only registered members with the `driver` role may enter, and the car must exist in the inventory.
- **Results:** Records finishing orders, updates driver rankings, and distributes prize money (60% for 1st, 30% for 2nd, 10% for 3rd) to the crew's cash balance. Updates car conditions if damaged.
- **Mission Planning:** Creates and manages missions (`delivery`, `rescue`, `recon`, `repair`, `heist`). Missions cannot start unless at least one crew member with the required role is available.

Additional Modules (Bonus)

- **Vehicle Upgrades Module:** Allows the admin to upgrade a car's speed or repair its condition using spare parts from the inventory (e.g., consuming an `engine` part gives +2 speed). Speed is capped at 10, and wrecked cars cannot be repaired.
- **Leaderboard Module:** Provides a ranked view of all drivers based on their race results. Drivers are ranked by wins first, then by total earnings. It aggregates data from the Results and Registration modules.

2.1 Call Graph

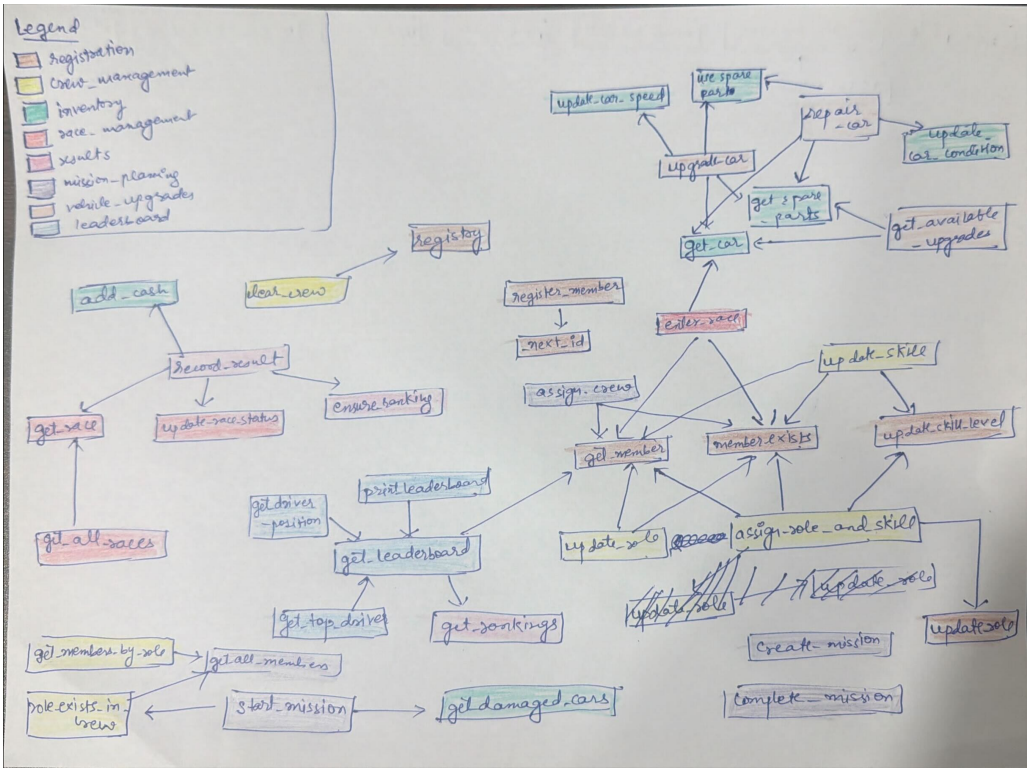

Figure 1: Call graph

2.2 Integration Test Design

The following 25 test cases validate how different modules interact with one another, based on the cross-module dependencies mapped out in the Call Graph. All tests are implemented in `tests/test_integration.py` and run using pytest.

- **TC-01: Register a member then assign a role**
  - **Why this test is needed:** This is the most fundamental cross-module interaction in the system. Crew Management must read from the Registration registry to assign a role, verifying this critical data flow.
  - **Scenario:** Register a member, then assign a role and skill level via Crew Management.
  - **Modules:** Registration, Crew Management
  - **Expected:** Member is registered with role `unassigned`. After assignment, the registry reflects the updated role and skill level.
  - **Actual:** PASSED
  - **Errors found:** None
- **TC-02: Assign a role to an unregistered member**
  - **Why this test is needed:** Business rules state a crew member must be registered before a role can be assigned. This verifies that Crew Management correctly enforces this rule by checking the Registration module.
  - **Scenario:** Attempt to assign a role to member ID 999 which does not exist.
  - **Modules:** Registration, Crew Management
  - **Expected:** `ValueError` raised with message "not registered".
  - **Actual:** PASSED
  - **Errors found:** None
- **TC-03: Register a driver and enter them into a race**
  - **Why this test is needed:** Covers the most common admin workflow and tests three cross-module interactions at once. Race Management must validate the driver via Registration and the car via Inventory.
  - **Scenario:** Register a member, assign driver role, add a car, create a race, and enter the driver.
  - **Modules:** Registration, Crew Management, Inventory, Race Management
  - **Expected:** Entry is created with the correct driver ID and car ID.
  - **Actual:** PASSED
  - **Errors found:** None
- **TC-04: Unregistered driver cannot enter a race**
  - **Why this test is needed:** Race Management calls `registration.member_exists()` before allowing entry. This verifies the check works and the race is protected from invalid entries.

- **Scenario:** Attempt to enter an unregistered member (ID 999) into a race.
- **Modules:** Registration, Race Management
- **Expected:** `ValueError` raised with message "not registered".
- **Actual:** PASSED
- **Errors found:** None

- **TC-05: Non-driver cannot enter a race**
  - **Why this test is needed:** Verifies the rule that only members with the `driver` role may enter a race, testing the validation interaction between Race Management and Registration.
  - **Scenario:** Register a member, assign mechanic role, attempt to enter them in a race.
  - **Modules:** Registration, Crew Management, Race Management
  - **Expected:** `ValueError` raised with message "only drivers".
  - **Actual:** PASSED
  - **Errors found:** None

- **TC-06: Race result updates cash balance**
  - **Why this test is needed:** Business rules dictate race results must update the cash balance. This tests the financial data transfer between Results and Inventory.
  - **Scenario:** Run a race with two drivers and record the result. Check that cash balance increased.
  - **Modules:** Race Management, Results, Inventory
  - **Expected:** Cash balance after the race is strictly greater than before.
  - **Actual:** PASSED
  - **Errors found:** None

- **TC-07: Race result updates driver rankings**
  - **Why this test is needed:** The Results module must update external rankings correctly after a race. This verifies wins and races counters are incremented successfully.
  - **Scenario:** Run a race and verify winner has 1 win and runner-up has 0 wins but 1 race.
  - **Modules:** Race Management, Results
  - **Expected:** Winner: wins=1, races=1. Runner-up: wins=0, races=1.
  - **Actual:** PASSED
  - **Errors found:** None

- **TC-08: Cannot record result for a race that has not started**
  - **Why this test is needed:** Results module calls Race Management to check the race status before recording. This verifies the cross-module status validation.

- **Scenario:** Create a race with entries but do not set it to ongoing. Attempt to record result.
- **Modules:** Race Management, Results
- **Expected:** `ValueError` raised with message "ongoing".
- **Actual:** PASSED
- **Errors found:** None

- **TC-09: Assign correct crew to a mission**
  - **Why this test is needed:** Mission Planning must validate crew roles via Registration before assigning them. This verifies the standard "happy path" where required roles are met.
  - **Scenario:** Create a rescue mission and assign a driver and a medic to it.
  - **Modules:** Registration, Crew Management, Mission Planning
  - **Expected:** Mission status remains `planned` with both members in assigned crew.
  - **Actual:** PASSED
  - **Errors found:** None

- **TC-10: Mission rejects crew with wrong roles**
  - **Why this test is needed:** Mission Planning checks that assigned crew collectively covers all required roles. This verifies the stringent role validation logic.
  - **Scenario:** Create a rescue mission (needs driver + medic) and assign only a driver.
  - **Modules:** Registration, Crew Management, Mission Planning
  - **Expected:** `ValueError` raised mentioning "medic" as unmet role.
  - **Actual:** PASSED
  - **Errors found:** None

- **TC-11: Mission cannot start without required role in crew**
  - **Why this test is needed:** Business rules state missions cannot start if required roles are unavailable. Tests interaction between Mission Planning and Crew Management at launch time.
  - **Scenario:** Create a recon mission (needs scout) and attempt to assign a driver to it.
  - **Modules:** Crew Management, Mission Planning
  - **Expected:** `ValueError` raised mentioning "scout" as missing role.
  - **Actual:** PASSED
  - **Errors found:** None

- **TC-12: Damaged car from race enables repair mission**

- **Why this test is needed:** Tests a complex four-module chain: A car damaged during a race must trigger state changes that a repair mission checks for before proceeding.
- **Scenario:** Run a race, mark a car as damaged in the result, then start a repair mission.
- **Modules:** Race Management, Results, Inventory, Mission Planning
- **Expected:** Car condition is set to `damaged` in inventory. Repair mission starts successfully.
- **Actual:** FAILED initially.
- **Errors found:** `record_result()` did not accept a `damaged_car_ids` parameter, preventing cars from being marked as damaged. **Fix:** Added the parameter to `record_result()` in the Results module to allow data transfer to Inventory.

• **TC-13: Repair mission blocked when no cars are damaged**

- **Why this test is needed:** A repair mission should only be allowed to start when there is actually something to repair. Tests validation between Mission Planning and Inventory.
- **Scenario:** Create a repair mission with a mechanic but no damaged cars in inventory.
- **Modules:** Inventory, Mission Planning
- **Expected:** `ValueError` raised with message "no damaged cars".
- **Actual:** FAILED initially.
- **Errors found:** `start_mission()` failed to check `inventory.get_damaged_cars()` before launch. **Fix:** Added a conditional check inside `start_mission()` in the Mission Planning module to halt execution if the damaged car list is empty.

• **TC-14: Upgrading a car consumes spare parts**

- **Why this test is needed:** Vehicle Upgrades must interact with Inventory to consume parts and mathematically update the car's speed attribute.
- **Scenario:** Add a car and an engine part, then apply an engine upgrade.
- **Modules:** Inventory, Vehicle Upgrades
- **Expected:** Car speed increases by 2. Engine part quantity decreases by 1.
- **Actual:** PASSED
- **Errors found:** None

• **TC-15: Upgrade blocked without spare parts**

- **Why this test is needed:** Vehicle Upgrades must check Inventory for parts before proceeding to prevent upgrading vehicles with items the crew does not possess.
- **Scenario:** Attempt to upgrade a car when no spare parts are in inventory.
- **Modules:** Inventory, Vehicle Upgrades

- **Expected:** `ValueError` raised mentioning the missing part.
- **Actual:** PASSED
- **Errors found:** None

- **TC-16: Leaderboard reflects race results**
  - **Why this test is needed:** Leaderboard pulls data from both Results (stats) and Registration (names). Verifies the full data flow from a completed race to a ranked visual output.
  - **Scenario:** Complete a race then check the leaderboard shows correct names, wins and order.
  - **Modules:** Race Management, Results, Registration, Leaderboard
  - **Expected:** Winner appears first with 1 win. Runner-up appears second with 0 wins.
  - **Actual:** PASSED
  - **Errors found:** None

- **TC-17: Update a member's role**
  - **Why this test is needed:** `update_role()` in Crew Management calls back into Registration to rewrite the registry. Verifies backwards data flow.
  - **Scenario:** Assign a driver role then update it to strategist.
  - **Modules:** Registration, Crew Management
  - **Expected:** Registry reflects the updated role `strategist`.
  - **Actual:** PASSED
  - **Errors found:** None

- **TC-18: Update a member's skill level**
  - **Why this test is needed:** `update_skill()` in Crew Management calls back into Registration to update the registry. Verifies continuous state synchronization.
  - **Scenario:** Assign a role with skill 5 then update skill to 9.
  - **Modules:** Registration, Crew Management
  - **Expected:** Registry reflects the updated skill level of 9.
  - **Actual:** PASSED
  - **Errors found:** None

- **TC-19: Filter crew members by role**
  - **Why this test is needed:** `get_members_by_role()` reads from the Registration registry to filter members. Verifies array filtering across module boundaries.
  - **Scenario:** Register two drivers and one mechanic, then filter by role.
  - **Modules:** Registration, Crew Management
  - **Expected:** `get_members_by_role("driver")` returns 2, `"mechanic"` returns 1.

- **Actual:** PASSED
- **Errors found:** None

- **TC-20: Repair a car consumes a repair kit**
  - **Why this test is needed:** `repair_car()` must interact with Inventory to mathematically deduct a repair kit and overwrite the car's condition string.
  - **Scenario:** Damage a car, add a repair kit to inventory, then repair the car.
  - **Modules:** Inventory, Vehicle Upgrades
  - **Expected:** Car condition becomes `good`. Repair kit quantity drops to 0.
  - **Actual:** PASSED
  - **Errors found:** None

- **TC-21: Get available upgrades based on spare parts**
  - **Why this test is needed:** `get_available_upgrades()` reads both car data and spare parts from Inventory. Verifies it correctly filters potential upgrades based on active stock.
  - **Scenario:** Add engine part and tyre to inventory (no nitrous kit) and check available upgrades.
  - **Modules:** Inventory, Vehicle Upgrades
  - **Expected:** Engine and tyres upgrades available. Nitrous not available.
  - **Actual:** PASSED
  - **Errors found:** None

- **TC-22: Get all races internally calls get_race**
  - **Why this test is needed:** Tests intra-module dependencies to verify that wrapper functions correctly aggregate data from base functions.
  - **Scenario:** Create two races and call `get_all_races()`.
  - **Modules:** Race Management
  - **Expected:** Returns a list of 2 races with correct names.
  - **Actual:** PASSED
  - **Errors found:** None

- **TC-23: Get top driver from leaderboard**
  - **Why this test is needed:** `get_top_driver()` calls `get_leaderboard()` internally which in turn calls Results and Registration. Verifies the entire aggregation chain.
  - **Scenario:** Complete a race and call `get_top_driver()`.
  - **Modules:** Results, Registration, Leaderboard
  - **Expected:** Returns the winning driver's name and 1 win.
  - **Actual:** PASSED

- **Errors found:** None

- **TC-24: Get a specific driver's leaderboard position**
  - **Why this test is needed:** `get_driver_position()` calls `get_leaderboard()` internally. Verifies the retrieval of a specific index from an aggregated cross-module list.
  - **Scenario:** Complete a race and get the runner-up's leaderboard position.
  - **Modules:** Results, Registration, Leaderboard
  - **Expected:** Runner-up is at position 2 with correct name.
  - **Actual:** PASSED
  - **Errors found:** None

- **TC-25: Print leaderboard produces correct output**
  - **Why this test is needed:** `print_leaderboard()` calls `get_leaderboard()` internally. Verifies the final UI/CLI output correctly formats the underlying data.
  - **Scenario:** Complete a race and call `print_leaderboard()`. Check printed output.
  - **Modules:** Results, Registration, Leaderboard
  - **Expected:** Printed output contains both driver names in the correct order.
  - **Actual:** PASSED
  - **Errors found:** None

Integration Testing Summary

Total Tests	Passed	Failed (Fixed)
25	23	2 (TC-12, TC-13)

Two integration tests failed on the first run due to cross-module communication gaps. TC-12 failed because ***`record_result()`*** *lacked the ability to pass damaged car data back to the Inventory. TC-13 failed because Mission Planning did not verify inventory states before launching. Both logical errors were fixed, bringing the final test suite to a 100% pass rate.*